

Credit Card Fraud Detection Database

Nicholas Perez

Tanner Lang

George Alenchery

COP 3710 | CRN: 14033

Overview



Database Scenario

This project uses the 'Credit Card Transactions Fraud Detection Dataset' consisting of synthetic credit card transactions from Kaggle.

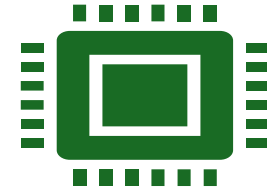
- The dataset included detailed information such as transaction amounts, locations, merchant details, and fraud indicators.

We designed the database schema with normalized tables based off the structure of the dataset and enforced data integrity through constraints.

- We did this with the intention that it would ultimately be used by our collaborators in Vietnam to train and evaluate machine learning models for detecting fraudulent transactions. (Although they have since stepped away from the project)

We populated our database using a custom python script designed to insert cleaned CSV data into each table.

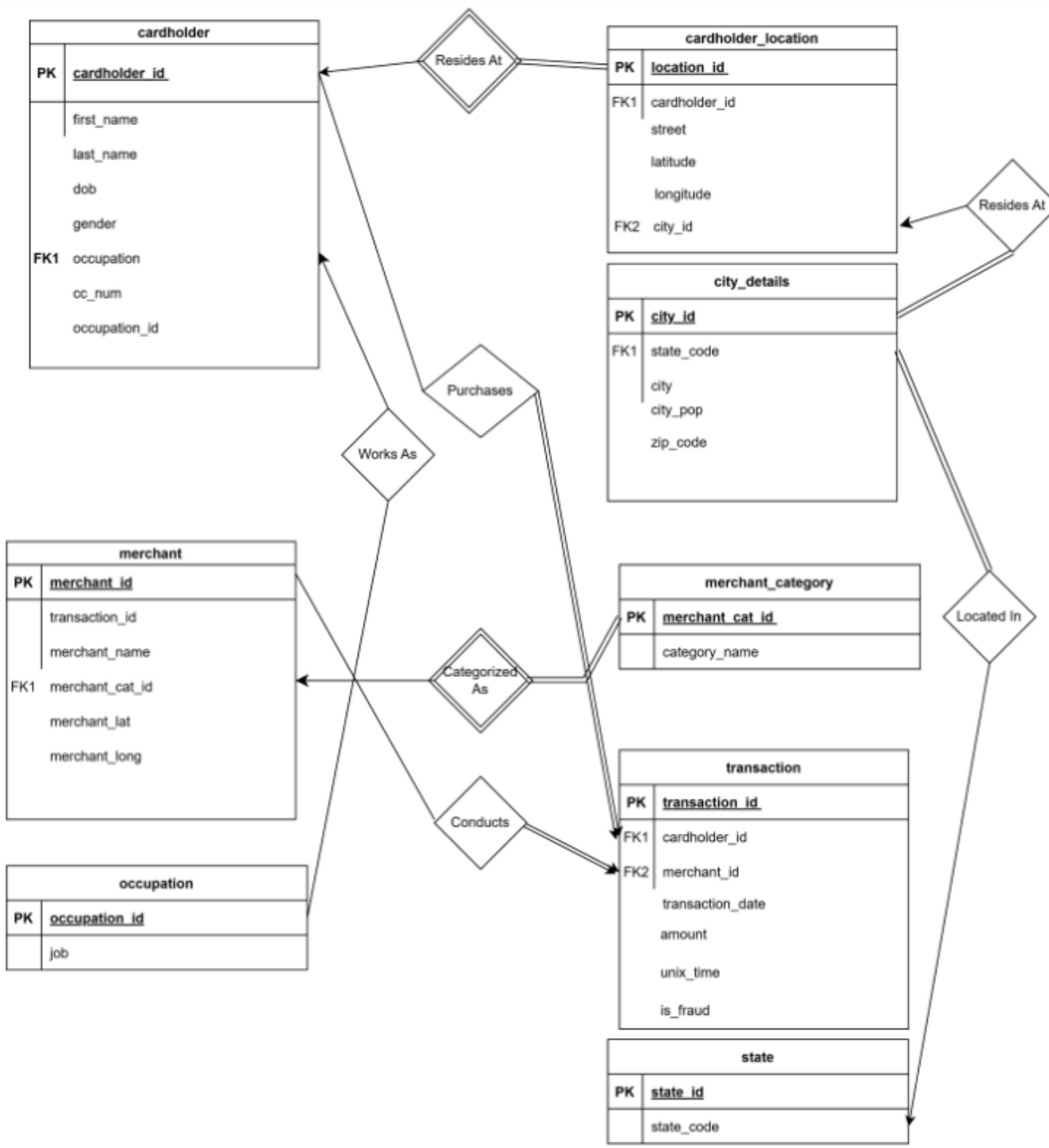
We created queries that vary in complexity to support the purpose of the database. We also connected the database to Jupyter Notebook for our collaborators and to visualizing the query data for the honor's contract.



Purpose

The purpose of our project was to build and populate a database capable of supporting fraud detection with queries of varying complexities. We also needed to be sure that database was normalized to reduce redundancy and ensure data integrity.

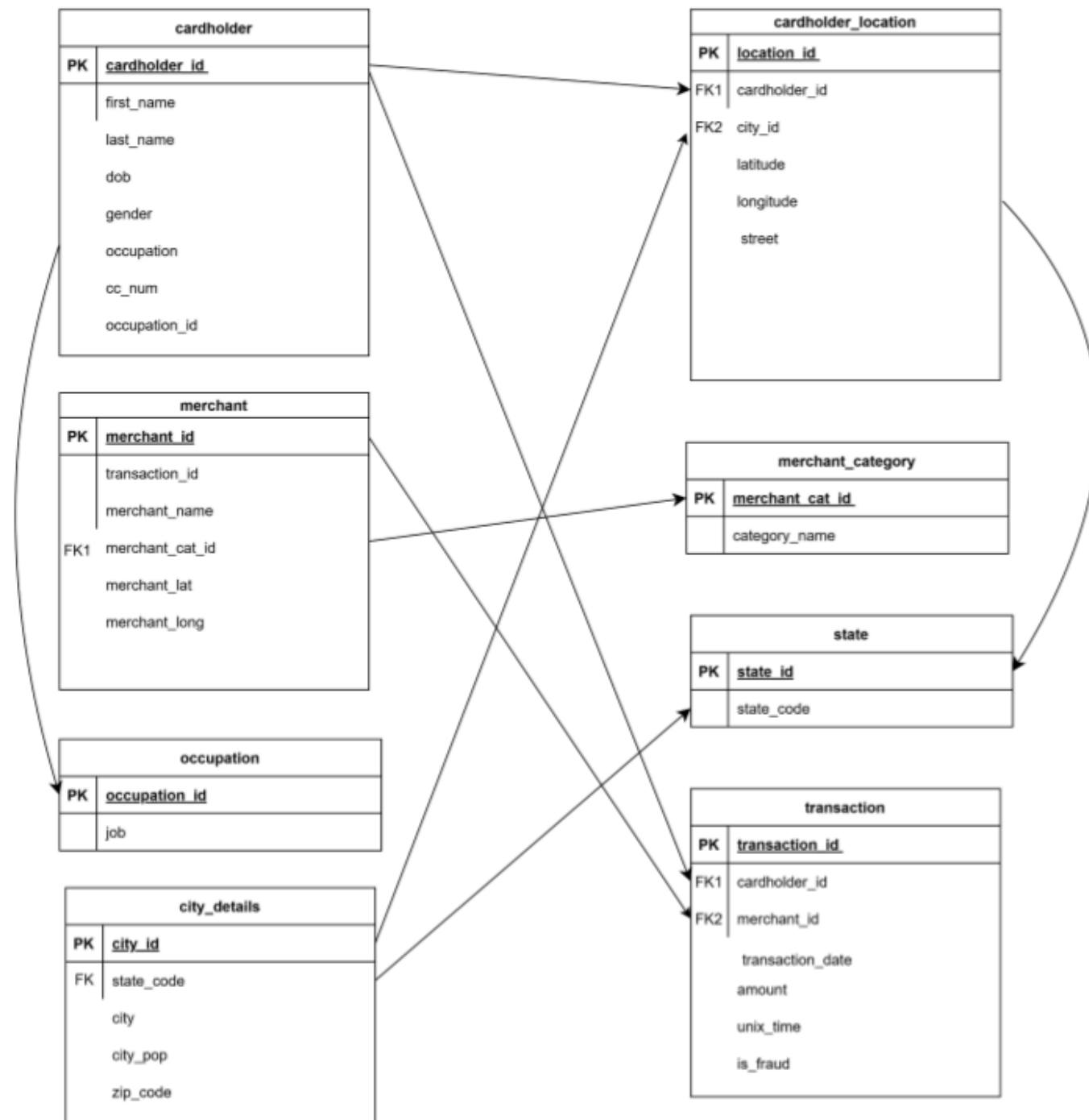
ER Diagram



- Our ER diagram models the major real-world objects and their relationships.
- Key Entities:
 - Cardholder, Merchant, Transactions, Occupation, States, Merchant Category, and Cardholder Location.
- Relationships
 - A cardholder can purchase multiple transactions. It requires total participation of the cardholder from purchases to transaction.
 - Each merchant belongs to a merchant category. The relationship here is called "Categorized As"
 - Each cardholder has one occupation and one address.
 - Each address is linked to a state.
 - A merchant can conduct multiple transactions. It requires total participation of the merchant from conducts to transactions
- The relationships use 1 to 1 or 1 to Many connections.
- The design supports fraud detection because it is a model to see who made what purchases, where, and when.

Schema Diagram

- Each entity was translated into a table with appropriate primary and foreign keys.
- Key Tables and Attributes
 - cardholder:** The central entity in the schema representing each card user. The cardholder_id is the primary key that serves as a foreign key in transactions and cardholder_location, linking personal and location data with transactions.
 - transactions:** Captures the individual transactions. It includes foreign keys cardholder_id and merchant_id which reference the cardholder and merchant tables, allowing fraud analysis based on user and merchant behavior.
 - merchant:** Represents the vendors involved in transactions. The merchant_id is the primary key and links to transactions. It contains a foreign key called merchant_cat_id which references the merchant_category table that was created as a result of 3NF normalization to avoid repeating category strings.
- Relationships are preserved with foreign keys to ensure referential integrity.
- We use 3NF normalization to remove redundancy and ensure data integrity
 - Remove repetitive data by converting some specific attributes into their own entities.
 - Made occupations into an occupation table.
 - Made merchant categories into a merchant_category table.
 - Created a separate cardholder_location table to eliminate redundancy.
 - It follows 3NF normalization since there are no partial or transitive dependencies.
- The schema supports complex queries for fraud detection.



Data Generation

- To generate the data, we utilized the power of AI and Python to create insertion statements based on our schema, tables, and constraints. We named the file `sql_populate.py` on our GitHub.
- The Python script created 51 insert statements for the 50 states plus the District of Columbia, which is part of the Kaggle dataset. Furthermore, the script created inserts for all tables by reading the CSV file with Pandas' function `read_csv()`. To make sure inserts were properly formatted and syntax was correct, apostrophes, commas, and more had to be removed prior to creating the insert statements.
- Because the dataset is massive (over 400mb of data), the amount of inserts had to be reduced to first 10,000 records, which equated to a little over 30,000 inserts. This is a big reduction from the original size of the insert file which contained well over two million inserts!





SQL Queries and Jupyter Notebook Data Visualization (Honors Contract)



- We aimed to design our queries to vary in complexity and to be relevant and effective for identifying fraudulent activity. Like:
 - View all transactions flagged as fraudulent
 - Detect merchants with >10% fraud rate
 - Identify users shopping at many merchants on the same day
 - We connected the database to Jupyter Notebook to enable access for our collaborators in Vietnam.
 - As mentioned before, they've since stepped away from the project.
 - Using python and matplotlib in Jupyter Notebook, I visualized the query result data with a variety of different charts.
 - Query explanation and data visualization demonstration in Jupyter Notebook:
- 

Conclusion

Data Engineering:

- We learned how to develop a database using SQL and how to query that database. We learned how to normalize tables and create constraints to uphold the integrity of the data.
- We learned how to create schema diagrams and ER diagrams to visualize the database design.

SQL and Querying:

- We learned how to use various functions like COUNT, FILTER, and SUM.
- We learned how to use GROUP and JOIN features.

